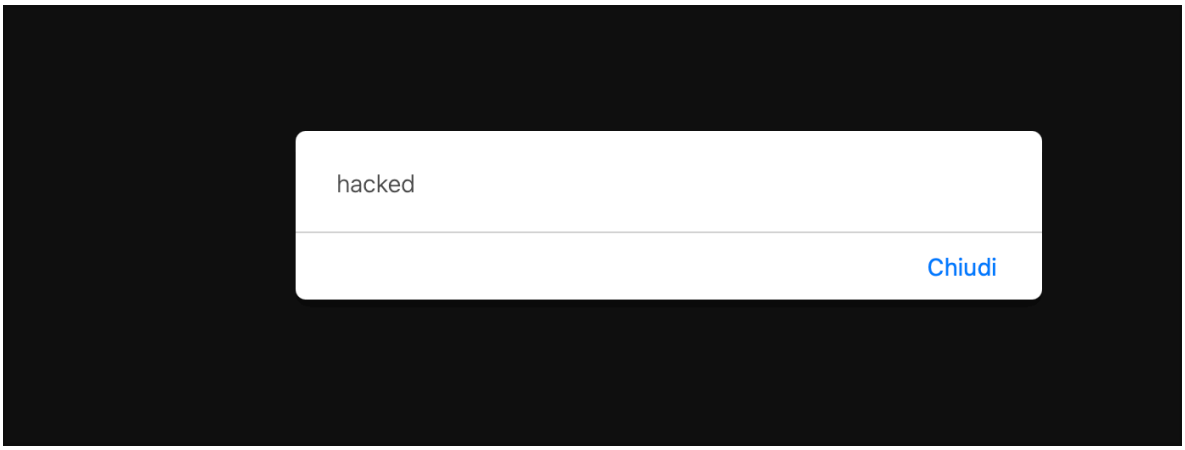
Challenge 5

Durante la creazione di un articolo si può inserire nel body del contenuto pericoloso. Se viene salvato e poi mostrato senza filtri, il browser lo esegue quando qualcuno apre l'articolo .
Nella vista show il body era stampato con {!! !!}, quindi senza escape automatico di Laravel.

Prova prima della mitigazione-

Come writer ho creato un articolo inserendo nel campo testo un payload di prova del PDF (script con alert('hacked')).
TinyMCE all'inizio toglieva quel contenuto, quindi, siccome non riuscivo a fare la prova come nel pdf, ho disattivato temporaneamente TinyMCE nel layout, usando una textarea normale perchè mi sembrava piu semplice e ha funzionato.
Aprendo l'articolo (cliccando Read) è comparso il popup "hacked".



Mitigazione-

Ho creato HtmlFilterService che pulisce l'HTML.
Poi l'ho usato in ArticleController:

- in store e update (prima di salvare)
- in show (anche in visualizzazione, per articoli già presenti)

Così il contenuto viene filtrato sia in salvataggio sia in lettura.

app > Services > HtmlFilterService.php

```php
<?php

namespace App\Services;

class HtmlFilterService
{
    public function filter(string $html): string
    {

        $clean = strip_tags($html, '<p><br><b><strong><i><em><ul><ol><li><a><h1><h2><h3>');


        $clean = preg_replace('/\s*on\w+\s*=\s*("|\')[^"\']*\1/i', '', $clean) ?? $clean;

        return $clean;
    }
}
```

> Http > Controllers > ArticleController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Tag;
use App\Models\User;
use App\Models\Article;
use App\Models\Category;
use App\Services\HtmlFilterService;
```

```php
public function store(Request $request, HtmlFilterService $htmlFilterService)
{
    $request->validate([
        'title' => 'required|unique:articles|min:5',
        'subtitle' => 'required|min:5',
        'body' => 'required|min:10',
        'image' => 'required|image',
        'category' => 'required',
        'tags' => 'required'
    ]);

    $article = Article::create([
        'title' => $request->title,
        'subtitle' => $request->subtitle,
        'body' => $htmlFilterService->filter($request->body),
        'image' => $request->file('image')->store('public/images'),
        'category_id' => $request->category,
        'user_id' => Auth::user()->id,
        'slug' => Str::slug($request->title),
```

> Controllers > ArticleController.php

```php
public function show(Article $article, HtmlFilterService $htmlFilterService)
{
    $article->body = $htmlFilterService->filter($article->body ?? '');
    return view('articles.show', compact('article'));
}

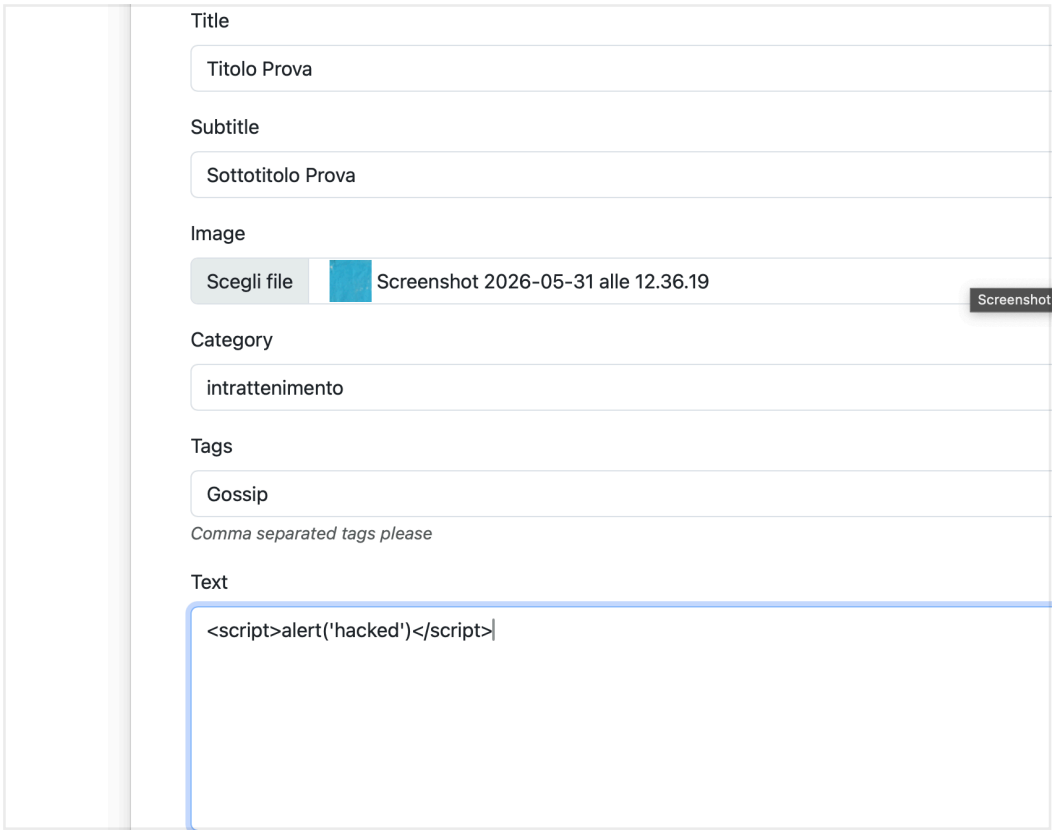
public function edit(Article $article)
{
    if(Auth::user()->id != $article->user_id){
        return redirect()->route('homepage')->with('alert', 'Accesso non consentito');
    }
    return view('articles.edit', compact('article'));
}

public function update(Request $request, Article $article, HtmlFilterService $htmlFilterService
{
    $request->validate([
        'title' => 'required|min:5|unique:articles,title,' . $article->id,
        'subtitle' => 'required|min:5',
        'body' => 'required|min:10',
        'image' => 'image',
        'category' => 'required',
        'tags' => 'required'
    ]);

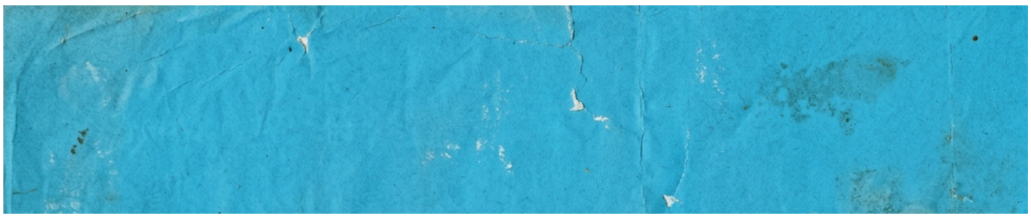
    $article->update([
        'title' => $request->title,
        'subtitle' => $request->subtitle,
        'body' => $htmlFilterService->filter($request->body),
```

Prova dopo la mitigazione-

Ho creato un nuovo articolo con lo stesso contenuto di prova. Aprendo la pagina non compare più il popup. Si vede solo il testo innocuo alert('hacked') (senza esecuzione).



Titolo Prova2



Sottotitolo Prova2

Category: **Intrattenimento**

Created at 03/10/2026 by Daria Richardson (Writer)

---

alert('hacked')

Go to article list